Ingegneria del Software – 27/08/2025 – Soluzione

Consegnare SOLO i fogli del tema.	**Attaccare l'etichetta in questa zona.**
Non separarli. Altri fogli saranno ignorati.	Compiti senza etichetta non sono validi.
I fogli sono fronte/retro. 2 ore di tempo.	
Si possono consultare libri e appunti.	
Tutti i dispositivi elettronici sono vietati.	

Esercizio 1

Si consideri la classe Java `SeasonManager` che gestisce il calendario delle partire per una stagione del campionato di calcio.

```
public class SeasonManager {
  // Ritorna l'insieme delle squadre che fanno parte del campionato
  public /*@ pure @*/ Set<Team> getTeams();

  // Ritorna la lista delle partite che compongono il campionato (il 'calendario'').
  // Un calendario e' valido  se ogni squadra affronta ogni altra squadra esattamente
  // due volte, una volta giocando in casa e una volta fuori casa.
  // Lancia una InsufficientTeamsException se non ci sono almeno 2 team.
  public /*@ pure @*/ List<Match> getMatches() throws InsufficientTeamsException;

  // Ritorna la classifica, contenente tutte le squadre in ordine decrescente di punteggio
  // sulla base della partire disputate fino a questo momento.
  // Una squadra guadagna 3 punti per ogni vittoria e 1 punto per ogni pareggio.
  public /*@ pure @*/ List<Team> getCurrentRanking();

  // Ritorna il risultato della partita passata come parametro
  public /*@ pure @ */ Result getResult(Match match);
}

public class Match {
  // Ritorna la squadra che gioca in casa in questa partita
  public /*@ pure @*/ Team getHomeTeam();

  // Ritorna la squadra che gioca fuori casa in questa partita
  public /*@ pure @*/ Team getAwayTeam();
}

public enum Result {
  HOME-WINS,  // Vince la squadra di casa
  AWAY-WINS,  // Vince la squadra fuori casa
  DRAW,       // Pareggio
  NOT-PLAYED  // Partita non ancora disputata
}
```

Domanda a) Si specifichi in JML il metodo `getMatches()`.

Soluzione

```
//@ensures getTeams().size() >= 2 && \result != null && !\result.contains(null) &&
//@ (\forall Match m; ; \result.contains(m) <==> (
//@   !m.getAwayTeam().equals(m.getHomeTeam()) &&
//@   getTeams().contains(m.getHomeTeam()) && getTeams().contains(m.getAwayTeam()) ) ) &&
//@ (\forall int i; i>=0 && i<\result.size();
//@   (\forall int j; j>=0 && j<i; !\result.get(i).equals(\result.get(j))) )
//@ signals (InsufficientTeamsException) getTeams().size() < 2
public /*@ pure @*/ List<Match> getMatches() throws InsufficientTeamsException;
```

Domanda b) Si specifichi in JML il metodo `getCurrentRanking()`.

Soluzione

Definiamo un predicato per indicare il punteggio di una squadra `t`, come segue.

```
teamPoints(t) ==
  (\numof Match m; getMatches().contains(m) && m.getHomeTeam().equals(t); getResult(m)==Result.HOME-WINS)*3 +
  (\numof Match m; getMatches().contains(m) && m.getAwayTeam().equals(t); getResult(m)==Result.AWAY-WINS)*3 +
  (\numof Match m; getMatches().contains(m) && m.getHomeTeam().equals(t); getResult(m)==Result.DRAW) +
  (\numof Match m; getMatches().contains(m) && m.getAwayTeam().equals(t); getResult(m)==Result.DRAW)
```

Possiamo quindi usare `teamPoints` per definire l'ordinamento delle squadre nella lista.

```
//@ ensures \result != null && !\result.contains(null) &&
//@ \result.containsAll(getTeams()) && \result.size() == getTeams().size() &&
//@ (\forall int i; i>=0 && \result.size()-1;
//@    teamPoints(\result.get(i)) >= teamPoints(\result.get(i+1)) )
public /*@ pure @*/ List<Team> getCurrentRanking();
```

Domanda c) Si consideri la seguente implementazione della classe `SeasonManager`, che fa uso di un insieme e due liste come indicato di seguito.

```
public class SeasonManager {
  private Set<Team> teams;
  private List<Match> matches;
  private List<Result> results;
  ...
}
```

L'insieme `teams` contiene l'elenco delle squadre, la lista `matches` contiene l'elenco delle partite, la lista `results` contiene l'elenco dei risultati. Data una partita in posizione `i` all'interno della lista `matches`, il suo risultato è contenuto in posizione `i` nella lista `results`.

Per tale implementazione si forniscano l'invariante di rappresentazione e la funzione di astrazione.

Soluzione

L'invariante di rappresentazione deve specificare che le collezioni non siano nulle e che non contengano valori nulli.

La lista `matches` deve contenere solo partite in cui le squadre sono diverse tra loro e fanno parte del campionato. Per ogni coppia di squadre `t1` e `t2`, deve contenere due partire in cui queste squadre si affrontano, una in cui `t1` gioca in casa e una in cui `t2` gioca in casa. Inoltre, la lista `matches` non deve contenere duplicati. Infine, la lista `results` deve avere la stessa dimensione della lista `matches`.

La lista `results` deve avere la stessa dimensione della lista `matches`.

```
//@ private invariant
//@ teams!=null && !teams.contains(null) &&
//@ matches!=null && !matches.contains(null) &&
//@ (\forall Match m; matches.contains(m); ! m.getHomeTeam().equals(m.getAwayTeam()) &&
//@   teams.contains(m.getHomeTeam()) && teams.contains(m.getAwayTeam()) ) &&
//@ (\forall int i; 0 <= i && i < matches.size();
//@   (\forall int j; 0 <= j && j < i; !matches.get(i).equals(matches.get(j))) ) &&
//@ (\forall Team t1; teams.contains(t1);
//@   (\forall Team t2; teams.contains(t2);
//@     (\exists Match m; matches.contains(m);
//@       m.getHomeTeam().equals(t1) && m.getAwayTeam().equals(t2)) &&
//@     (\exists Match m; matches.contains(m);
//@       m.getHomeTeam().equals(t2) && m.getAwayTeam().equals(t1)) ) ) &&
//@ results!=null && !results.contains(null) && results.size()==matches.size();
```

La funzione di astrazione deve definire i metodi `getTeams()` e `getResult()` a partire dalle liste `teams`, `matches` e `results`. Tutti gli altri metodi possono infatti essere definiti a partire da questi.

```
//@ private invariant
//@ (\forall Team t; ; getTeams().contains(t) <==> teams.contains(t)) &&
//@ (\forall Match m; ; getMatches().contains(m) <==> matches.contains(m)) &&
//@ (\forall int i; i>=0 && i<matches.size();
//@   (getResult(matches.get(i)) == results.get(i) )
```

Domanda d) Si consideri una classe `LargeSeasonManager` per campionati di grosse dimensioni. La classe modifica il metodo `getMatches()` per lanciare una `InsufficientTeamsException` se non ci sono almeno 16 teams nel campionato. È possibile definire `LargeSeasonManager` come sottoclasse di `SeasonManager` in accordo con il principio di sostituzione di Liskov? Si motivi la propria risposta.

Soluzione

Non è possibile definire `LargeSeasonManager` come sottoclasse di `SeasonManager`, in quanto modifica le post-condizioni nel caso di campionati con almeno 2 ma meno di 16 squadre. In questo caso `getMatches()` in `LargeSeasonManager` lancia una eccezione, e questa postcondizione eccezionale non è compatibile (ovvero, più restrittiva) con la postcondizione dello stesso metodo nella classe `SeasonManager`.

Esercizio 2

Si consideri questa variante della classe `SeasonManager`:

```
class SeasonManager {
  private final List<Match> matches = new ArrayList<Match>();

  // Aggiunge una partita al campionato
  public void addMatch(Match m) {
    synchronized (matches) {
      matches.add(m);
    }
    synchronized (this) {
      notifyAll();
    }
  }

  // Attende finche' la lista delle partite non contiene almeno una partita
  public void waitForMatch() throws InterruptedException {
    synchronized (this) {
      while (matches.isEmpty()) {
        wait();
      }
    }
  }

  // Restituisce la lista delle partite
  public synchronized List<Match> getMatches() {
    return matches;
  }
}

class Match {
  private final Team home;
  private final Team away;
  public Match(Team h, Team a) { home=h; away=a; }
  public Team getHomeTeam() { return home; }
  public Team getAwayTeam() { return away; }
}

class Team {
  private final String name;
  public Team(String n) { name = n; }
  public String getName() { return name; }
}
```

Domanda a) La classe non è correttamente sincronizzata. Come mai?

Il problema principale è che vengono usati due lock diversi:

- `addMatch` sincronizza su `matches`,
- `waitForMatch` sincronizza su `this`.

Questo può far sì che un thread rimanga bloccato in `waitForMatch()` anche se un'altra partita è già stata aggiunta, perché l'aggiunta di una nuova partita è su un lock diverso rispetto alla lettura di `matches`.

Domanda b) Si descriva un esempio di scenario con due thread che può dare luogo a un comportamento scorretto, accedendo ad una stessa variabile `SeasonManager sm = new SeasonManager();` e lo si implementi in Java (si mostri il frammento di codice che crea e avvia i due thread che con il loro codice, accedendo al medesimo `SeasonManager sm`, danno luogo al problema evidenziato al punto precedente.)

Esempio:

- Thread T1 esegue `addMatch(new Match(team1, team2))` → aggiunge la partita sotto lock su `matches` e fa `notifyAll()` su `this`.
- Thread T2 esegue `waitForMatch()` → entra nel blocco sincronizzato su `this` e legge `matches.isEmpty()` senza lock coerente, può quindi chiamare `wait()` pur se la partita è già presente (aggiunta nel tempo tra la `matches.isEmpty()` e la `wait`).

```
SeasonManager sm = new SeasonManager();
Team team1 = new Team("A");
Team team2 = new Team("B");

// Thread 1: aggiunge una partita
Thread t1 = new Thread(() -> { sm.addMatch(new Match(team1, team2)); });
```

```
        // Thread 2: attende una partita
        Thread t2 = new Thread(() -> {
                try {
                        sm.waitForMatch();
                        System.out.println("Partita trovata!");
                } catch (InterruptedException e) { }
        });

        t2.start();
        t1.start();
```

Domanda c) Anche risolvendo il problema di sincronizzazione del punto a), il codice presente ancora un problema potenziale di conflitti di sincronizzazione. Quale?

Soluzione: Il problema è che `getMatches()` restituisce direttamente la lista condivisa `matches`.

- Un thread esterno può modificarla senza sincronizzazione.
- Iterazioni sulla lista possono dare `ConcurrentModificationException` se un altro thread modifica contemporaneamente la lista.

La soluzione corretta sarebbe restituire una copia della lista, ad esempio `new ArrayList<>(matches)`.

Esercizio 3

Si consideri il seguente metodo statico.

```
public static int analizzaValori(int limite, int soglia) {
  if (limite <= 0) {
    return -1;
  }
  int contatore = 0;
  for (int i = 0; i < limite; i++) {
    if (i > soglia && i % 2 != 0) {
      contatore++;
    }
  }
  return contatore;
}
```

Domanda a) Si definisca un insieme minimo di casi di test per coprire tutte le istruzioni del metodo (statement coverage). Si motivi la propria risposta.

Soluzione

Sono sufficienti due casi di test.

Un caso entra nel primo `if`, ad esempio (`limite=0`, `soglia` qualsiasi).

Un secondo caso entra nel `for` e poi nel secondo `if`, ad esempio (`limite=2`, `soglia=0`).

Domanda b) Si definisca un insieme minimo di casi di test per coprire tutte le decisioni del metodo (branch coverage). Si motivi la propria risposta.

Soluzione

I casi precedenti sono sufficienti.

Il primo caso copre il ramo `true` del primo `if`.

Il secondo caso copre il ramo `false` del primo `if`, nonchè il ramo `false` del secondo `if` alla prima iterazione (in quanto non risulta vero `i>soglia`), e il ramo `true` alla seconda iterazione (quando `i` vale `1` ed è quindi maggiore di `soglia` e dispari).

Domanda c) Si definisca un insieme minimo di casi di test per coprire tutte le decisioni e le condizioni del metodo (branch and condition coverage). Si motivi la propria risposta.

Soluzione

Con riferimento al secondo `if`, i casi precedenti coprono il caso in cui `i>soglia` è `false` e il caso in cui entrambe le condizioni sono `true`. Manca un caso in cui la seconda condizione è `false`. Per ottenerla è sufficiente modificare il secondo caso di test, aumentando il `limite`, ad esempio (`limite=3`, `soglia=0`). In questo modo, si potrà ottenere `i=2`, che rende vera la prima condizione e falsa la seconda.

In definitiva, sono ancora sufficienti due casi di test.

Esercizio 4

Si considerino le seguenti classi Java:

```
abstract class Personaggio {
  protected double puntiForza;

  public Personaggio(double puntiForza) {
    this.puntiForza = puntiForza;
  }
  public void assorbiForza(Personaggio altro) {
    this.puntiForza = this.puntiForza + altro.puntiForza;
  }
  public abstract void combattiEStampa(Personaggio altro);
}

class Guerriero extends Personaggio {
  public Guerriero(double puntiForza) {
    super(puntiForza);
  }
  public void combattiEStampa(Personaggio altro) {
    this.assorbiForza(altro);
    System.out.println("Forza finale Guerriero: " + this.puntiForza);
  }
}

class Mago extends Personaggio {
  public Mago(double puntiForza) {
    super(puntiForza);
  }
  public void combattiEStampa(Personaggio altro) {
    this.assorbiForza(altro);
      // I maghi ottengono un bonus incantando l'avversario
      System.out.println("Forza finale Mago con bonus magico: " + (this.puntiForza + 20.0));
  }
}

class Cavaliere extends Guerriero {
  public Cavaliere(double puntiForza) {
    super(puntiForza);
  }
  public void combattiEStampa(Guerriero altroGuerriero) {
    // Bonus per sinergia in combattimento
    this.puntiForza = this.puntiForza + altroGuerriero.puntiForza + 15.0;
    System.out.println("Forza finale Cavaliere con onore: " + this.puntiForza);
  }
}
```

e il seguente frammento di codice le cui righe sono state numerate per riferimento:

```
1  Personaggio p1 = new Guerriero(80.0);
2  Personaggio p2 = new Mago(70.0);
3  Guerriero g1 = new Guerriero(80.0);
4  Mago m1 = new Mago(70.0);
5  Cavaliere c1 = new Cavaliere(100.0);
6  Cavaliere c2 = new Cavaliere(100.0);
7  Mago m2 = (Mago) new Guerriero(50.0);
8
9  p1.combattiEStampa(p2);
10 p2.combattiEStampa(g1);
11 g1.combattiEStampa(m1);
12 m1.combattiEStampa(m2);
13 c1.combattiEStampa(g1);
14 c2.combattiEStampa(m1);
```

Domanda a) Indicare le istruzioni che generano errore in compilazione o run-time, specificando il tipo di errore e la motivazione.

Soluzione

La riga 7 genera un errore in fase di compilazione, in quanto si sta tentando di forzare una conversione di tipo tra `Guarriero` e `Mago`, tra cui non sussiste una relazione di ereditarietà. Di conseguenza, anche la riga 12 (che fa uso di `m2`) non può essere eseguita.

Domanda b) Avendo eliminato le righe con errore, scrivere cosa viene stampato per ogni riga rimanente.

Soluzione

09. Forza finale Guerriero: 150.0
10. Forza finale Mago con bonus magico: 170.0
11. Forza finale Guerriero: 150.0
13. Forza finale Cavaliere con onore: 265.0
14. Forza finale Guerriero: 170.0